РАЗРАБОТКА НА C#

Unit-тестирование в C#

[Fact]
[Theory]

Пирамида тестов, best practices, TDD, xUnit, Assertions, моки и тестовые данные

Юрий Уфимцев

О чём поговорим

01

Тестирование

цели и пирамида тестов

02

Unit-тесты

заглушки и best practices

03

TDD

разработка через тесты

04

Unit-тесты в C#

фреймворки и xUnit

05

Assertions

Assert и FluentAssertions

06

Моки

Moq и NSubstitute

07

Тестовые данные

Builder, Bogus, AutoFixture

01

Тестирование

Зачем тестировать и как устроена пирамида тестов

01 · ТЕСТИРОВАНИЕ

Что такое тестирование?

Тестирование ПО - процесс испытания программного продукта, имеющий различные цели.

- Обнаружение дефектов
- Уверенность в уровне качества
- Информация для принятия решений
- Предотвращение дефектов
- Демонстрация соответствия требованиям

01 · ТЕСТИРОВАНИЕ

Пирамида тестирования



Снизу вверх

Разработка ПО - движение по пирамиде снизу вверх.

Выше - дороже

Чем выше уровень, тем тесты сложнее, дороже и медленнее; дольше ищется регресс.

Без дублирования

Тесты верхнего уровня не перепроверяют нижние.

01 · ТЕСТИРОВАНИЕ

Уровни автотестов

Unit

- Методы и классы
- Очень простые и быстрые
- Их больше всего
- Не зависят от других модулей
- Быстро показывают регресс

Интеграционные

- Взаимодействие компонентов
- Внешние зависимости: БД, сеть, другие сервисы

End-to-end

- Всё приложение целиком
- Сценарии пользователя
- Часто пишут QA-инженеры
- Пишут, когда продукт достиг нужного качества

02

Unit-тесты

Что это, какие бывают заглушки и как писать тесты хорошо

Unit-тесты

Unit-тестирование - проверка работоспособности отдельных функциональных модулей, процессов или частей кода приложения.

Что считается «юнитом»:

- **метод**
- **объект**
- **модуль**
- **процедура**
- **функция**

02 · UNIT-ТЕСТЫ

Для чего нужны unit-тесты

1

Ранний поиск ошибок

Ошибки находятся и исправляются на ранних этапах - это снижает расходы.

2

Понимание кода

Помогают понять базовый код, упрощают чтение и доработку проекта.

3

Живая документация

Новичок в проекте разберётся в принципах его работы по тестам.

Плюсы и минусы unit-тестов

Плюсы

- Простота
- Высокая информативность
- Повторное применение
- Поддержка параллельной разработки

Минусы

- Нет гарантии, что найдутся все ошибки
- Ориентированность на одну единицу кода (модуль)

Типы заглушек в тестировании

1. Фиктивные объекты (Dummy objects)

- Передаются классу, но никогда не используются
- Применяются, когда нужно заполнить параметры, не важные для теста

2. Фальшивые объекты (Fake objects)

- Имеют реальную работающую реализацию, но упрощенную
- Пример: фальшивая база данных на основе HashMap
- Распространенный пример: H2 DB - in-memory база для тестирования
    i. Чаще используется Testcontainers с реальной БД поднятой в docker

3. Заглушки (Stubs)

- Возвращают предопределенные ответы на вызовы
- Не имеют реальной реализации
- Самый популярный тип заглушки

Типы заглушек в тестировании

4. Макеты (Mocks)

Работают как заглушки, но также отслеживают взаимодействия

- Позволяют проверять, сколько раз и с какими параметрами вызывался метод
- Пример: проверка, что getAllInvoices вызывался ровно один раз
- Популярны для анализа взаимодействия между классами

5. Шпионы (Spies)

- "Обертывают" реальный объект и наблюдают за его поведением
- Не имитируют объект, а записывают взаимодействия с ним
- Используются, когда проще работать с реальной реализацией

Best practices

1 Наименование тестов

2 Arrange · Act · Assert

3 Minimally passing tests

4 Без «волшебных» строк

5 Без логики в тестах

6 Один тест - один кейс

7 Приватное - через публичное

8 Заглушки для статики

BEST PRACTICE 1 / 8

Наименование тестов

Длинное имя теста - это нормально: главное, чтобы оно описывало проверяемый сценарий.

`Add` `_SingleNumber` `_ReturnsSameNumber`

тестируемый метод | сценарий | ожидаемый результат

ПЛОХО

```
[Fact]
public void Test_Single()
{
    var stringCalculator = new StringCalculator();

    var actual = stringCalculator.Add("0");

    Assert.Equal(0, actual);
}
```

ЛУЧШЕ

```
[Fact]
public void Add_SingleNumber_ReturnsSameNumber()
{
    var stringCalculator = new StringCalculator();

    var actual = stringCalculator.Add("0");

    Assert.Equal(0, actual);
}
```

BEST PRACTICE 2 / 8

Организация тестов: AAA

Arrange
создание и настройка объектов

Act
действие над объектом

Assert
проверка результата

ПЛОХО

```
[Fact]
public void Add_EmptyString_ReturnsZero()
{
    // Arrange
    var stringCalculator = new StringCalculator();

    // Assert
    Assert.Equal(0, stringCalculator.Add(""));
}
```

ЛУЧШЕ

```
[Fact]
public void Add_EmptyString_ReturnsZero()
{
    // Arrange
    var stringCalculator = new StringCalculator();
    // Act
    var actual = stringCalculator.Add("");
    // Assert
    Assert.Equal(0, actual);
}
```

BEST PRACTICE 2 / 8

Аналог AAA: Given - When - Then

Тот же шаблон в терминах BDD (Behavior-Driven Development). Встречается в комментариях и в именах тестов.

Arrange	→	**Given** — дано: исходное состояние
Act	→	**When** — когда: выполняем действие
Assert	→	**Then** — тогда: ожидаемый результат

`Given_EmptyString_When_Add_Then_Zero`

```
[Fact]
public void Add_EmptyString_ReturnsZero()
{
    // Given
    var stringCalculator = new StringCalculator();
    // When
    var actual = stringCalculator.Add("");
    // Then
    actual.Should().Be(0);
}
```

BEST PRACTICE 3 / 8

Minimally passing tests

Входные данные - максимально простые, ровно чтобы проверить поведение. Чем сложнее данные, тем выше шанс ошибиться в самом тесте.

ПЛОХО

```
[Fact]
public void Add_SingleNumber_ReturnsSameNumber()
{
    var stringCalculator = new StringCalculator();

    var actual = stringCalculator.Add("42");

    Assert.Equal(42, actual);
}
```

ЛУЧШЕ

```
[Fact]
public void Add_SingleNumber_ReturnsSameNumber()
{
    var stringCalculator = new StringCalculator();

    var actual = stringCalculator.Add("0");

    Assert.Equal(0, actual);
}
```

BEST PRACTICE 4 / 8

Избегайте «волшебных» строк

Именование в тестах не менее важно, чем в рабочем коде: значение должно объяснять, *почему* оно такое.

ПЛОХО

```
[Fact]
public void Add_BigNumber_ThrowsException()
{
    var stringCalculator = new StringCalculator();

    Action actual =
        () ⇒ stringCalculator.Add("1001");

    Assert.Throws<OverflowException>(actual);
}
```

ЛУЧШЕ

```
[Fact]
void Add_MaximumSumResult_ThrowsOverflowException()
{
    var stringCalculator = new StringCalculator();
    const string MAX = "1001";

    Action actual =
        () ⇒ stringCalculator.Add(MAX);

    Assert.Throws<OverflowException>(actual);
}
```

BEST PRACTICE 5 / 8

Избегайте логики в тестах

Никаких **if, while, for, switch**. Вместо цикла по кейсам - параметризованный тест.

ПЛОХО

```
[Fact]
public void Add_MultipleNumbers_ReturnsCorrectResults()
{
    var stringCalculator = new StringCalculator();
    var expected = 0;
    var testCases =
        new[] { "0,0,0", "0,1,2", "1,2,3" };

    foreach (var test in testCases)
    {
        Assert.Equal(expected,
            stringCalculator.Add(test));
        expected += 3;
    }
}
```

ЛУЧШЕ

```
[Theory]
[InlineData("0,0,0", 0)]
[InlineData("0,1,2", 3)]
[InlineData("1,2,3", 6)]
public void Add_MultipleNumbers_ReturnsSumOfNumbers(
    string input, int expected)
{
    var stringCalculator = new StringCalculator();

    var actual = stringCalculator.Add(input);

    Assert.Equal(expected, actual);
}
```

BEST PRACTICE 6 / 8

Один тест - один кейс

Лучше два похожих теста, чем всё в одном: отдельный тест на действие или параметризованный **[Theory]**.

ПЛОХО

```
[Fact]
public void Add_EmptyEntries_ShouldBeTreatedAsZero()
{
    // Act
    var actual1 = stringCalculator.Add("");
    var actual2 = stringCalculator.Add(",");

    // Assert
    Assert.Equal(0, actual1);
    Assert.Equal(0, actual2);
}
```

ЛУЧШЕ

```
[Theory]
[InlineData("", 0)]
[InlineData(",", 0)]
public void Add_EmptyEntries_ShouldBeTreatedAsZero(
    string input, int expected)
{
    // Arrange
    var stringCalculator = new StringCalculator();
    // Act
    var actual = stringCalculator.Add(input);
    // Assert
    Assert.Equal(expected, actual);
}
```

BEST PRACTICE 7 / 8 · ПРИВАТНОЕ - ЧЕРЕЗ ПУБЛИЧНОЕ

В большинстве случаев приватный метод тестировать не нужно.

Важен конечный результат публичного метода, который этот приватный метод вызывает.

BEST PRACTICE 8 / 8

Заглушки для статических ссылок

Тест должен полностью контролировать систему. Статические вызовы вроде **DateTime.Now** это ломают.

```
public int GetDiscountedPrice(int price)
{
    if (DateTime.Now.DayOfWeek == DayOfWeek.Tuesday)
    {
        return price / 2;
    }
    else
    {
        return price;
    }
}
```

Проблема

Тест на вторник проходит только по вторникам, тест на остальные дни - во все дни, кроме вторника.

Решение

Передавать время в метод снаружи: провайдер или DateTimeOffset вместо DateTime.Now.

BEST PRACTICE 8 / 8

Решение: абстракция над временем

```
public interface IDateTimeProvider
{
    DayOfWeek DayOfWeek();
}

public int GetDiscountedPrice(int price, IDateTimeProvider dateTimeProvider)
{
    if (dateTimeProvider.DayOfWeek() == DayOfWeek.Tuesday)
    {
        return price / 2;
    }
    else
    {
        return price;
    }
}
```

Зависимость снаружи

Метод больше не обращается к DateTime.Now - время приходит параметром.

В тесте

Подставляем заглушку с нужным днём недели.

BEST PRACTICE 8 / 8

Тесты с заглушкой (Moq)

```
public void GetDiscountedPrice_NotTuesday_ReturnsFullPrice()
{
    var priceCalculator = new PriceCalculator();
    var dateTimeProviderStub = new Mock<IDateTimeProvider>();
    dateTimeProviderStub.Setup(dtp ⇒ dtp.DayOfWeek()).Returns(DayOfWeek.Monday);
    var actual = priceCalculator.GetDiscountedPrice(2, dateTimeProviderStub.Object);
    Assert.Equal(2, actual);
}

public void GetDiscountedPrice_OnTuesday_ReturnsHalfPrice()
{
    var priceCalculator = new PriceCalculator();
    var dateTimeProviderStub = new Mock<IDateTimeProvider>();
    dateTimeProviderStub.Setup(dtp ⇒ dtp.DayOfWeek()).Returns(DayOfWeek.Tuesday);
    var actual = priceCalculator.GetDiscountedPrice(2, dateTimeProviderStub.Object);
    Assert.Equal(1, actual);
}
```

Практические советы

Детерминированность

Одинаковый результат при каждом запуске - без случайных данных.

Покрытие ≠ качество

Высокий процент покрытия не говорит о качестве кода.

Длинные имена - ок

Главное, чтобы имя отражало проверяемый сценарий.

Что тестировать

Прежде всего бизнес-логику. БД unit-тестами не проверяют.

Асинхронный код

Почти как синхронный: ReturnsAsync вместо Returns.

Тесты улучшают дизайн

Неудобно тестировать - повод пересмотреть абстракции.

03

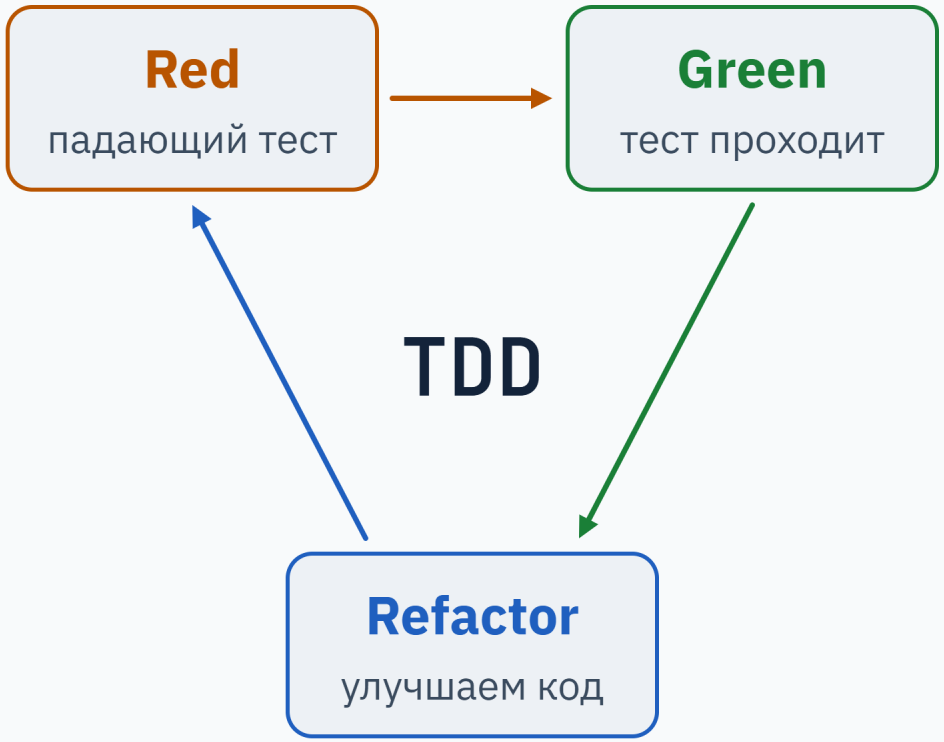
Test-Driven Development

Разработка через тестирование: цикл, процесс, плюсы и минусы

TDD

Техника разработки на основе повторения очень коротких циклов:

1. Пишем тест на желаемое изменение
2. Пишем код, чтобы тест прошёл
3. Рефакторим новый код под стандарты



Процесс TDD

1. Пишет тест для новой функции по спецификации
2. Запускает все существующие тесты
3. Пишет простейший код, чтобы тест прошёл
4. Рефакторит код: читаемость и сопровождение
5. Повторяет процесс для каждой функции

Плюсы и минусы TDD

Плюсы

- Надёжный код - быстрее поставка
- Код гибкий: рефакторинг с минимальным риском
- Тесты сами проверены: каждый сначала падает - минимум ложных срабатываний
- Код легко тестировать
- Нет лишнего кода - только под требования

Минусы

- Больше времени на код и тесты
- Нужен большой опыт: плохие тесты обесценивают подход
- Некоторые тесты сложно написать - например, для GUI

TDD и тесты на практике

ТИПИЧНЫЙ НАБОР ТЕСТОВ

1 интеграционный тест
на каждую ручку (эндпоинт)

Тесты бизнес-логики
1 успешный и все неуспешные кейсы

E2E-тесты
на пользовательские сценарии

- TDD не обязателен: многие команды работают по другим методологиям.
- Старт по TDD: интерфейсы с пустой реализацией, затем тесты.
- Высокий % покрытия не обязателен; много багов - покрытие растят.
- Хрупкие тесты всплывают при рефакторинге.

04

Unit-тесты в C#

Фреймворки, атрибуты xUnit и общий контекст

Фреймворки тестирования

xUnit.net

Создан одним из авторов NUnit, исправляет его недостатки. Раньше вышел на .NET Core. Каждый тест - в своём экземпляре класса.

NUnit

Исторически один экземпляр тестового класса на все тесты. Сейчас тоже умеет изолировать контекст.

MSTest

Раньше был проприетарным, сейчас open source и активно развивается.

На практике используют все три, но популярнее всего **xUnit**.

Основные понятия xUnit

`[Fact]`	Отдельный юнит-тест без параметров.
`[Theory]`	Тест с параметрами: один код - несколько сценариев.
`Fixture`	Класс для настройки и очистки контекста. Подключается через IClassFixture или через коллекцию и ICollectionFixture - тогда контекст общий для нескольких классов.

04 · XUNIT

[Fact]

Атрибут **[Fact]** объявляет метод теста, который выполняется средством запуска тестов.

Запуск - из IDE или в консоли: dotnet test

```
using Xunit;
using Prime.Services;

namespace Prime.UnitTests.Services
{
    public class PrimeService_IsPrimeShould
    {
        [Fact]
        public void IsPrime_ReturnFalse()
        {
        }
    }
}
```

[Theory] + [InlineData]

[Theory] - набор тестов: один код, разные входные аргументы.

[InlineData] задаёт значения аргументов - каждый набор становится отдельным тестом.

```
[Theory]
[InlineData(-1)]
[InlineData(0)]
[InlineData(1)]
public void IsPrime_ValuesLessThan2_ReturnFalse(int value)
{
    var result = _primeService.IsPrime(value);

    Assert.False(result, $"{value} should not be prime");
}
```

[Theory] + [MemberData]

[MemberData] берёт наборы аргументов из статического члена класса.

Удобно для значений enum и сложных наборов данных.

```
public class TestSuite
{
    [Theory]
    [MemberData(nameof(AgeHeightWeightData))]
    public void Should_measure_weight(
        int age, int height, decimal weight) { ... }

    public static IEnumerable<object[]> AgeHeightWeightData()
    {
        yield return new object[] { 20, 180, 80 };
        yield return new object[] { 20, 180, 50 };
    }
}
```

Setup и teardown: конструктор и Dispose

xUnit создаёт **новый экземпляр класса на каждый тест** - у каждого теста свой Stack.

Конструктор - повторяющаяся настройка. Dispose - очистка.

```
public class StackTests : IDisposable
{
    Stack<int> stack;
    public StackTests()
    {
        stack = new Stack<int>();
    }
    public void Dispose()
    {
        stack.Clear();
    }
    [Fact]
    public void AfterPushingItem_CountShouldReturnOne()
    {
        stack.Push(42);
        var count = stack.Count;
        Assert.Equal(1, count);
    }
}
```

04 · XUNIT

IClassFixture - общий контекст класса

Фикстура создаётся **один раз** на все тесты класса - они видят изменения друг друга.

xUnit сам создаст фикстуру и передаст в конструктор. Фикстур может быть несколько.

Примеры: БД для интеграционных тестов, WebApplicationFactory.

```
public class SharedInMemoryDbContextTests
    : IClassFixture<InMemoryDbContextFixture>
{
    InMemoryDbContextFixture fixture;
    public SharedInMemoryDbContextTests(InMemoryDbContextFixture fixture)
    {
        this.fixture = fixture;
    }
    [Fact]
    public void WithNoItems_CountShouldReturnZero()
    {
        var count = fixture.Context.Users.Count();
        Assert.Equal(0, count);
    }
    [Fact]
    public void AfterAddingItem_CountShouldReturnOne()
    {
        fixture.Context.Users.Add(new User());
        var count = fixture.Context.Users.Count();
        Assert.Equal(1, count);
    }
}
```

ICollectionFixture - контекст на несколько классов

[CollectionDefinition] + ICollectionFixture<T>, а тестовые классы помечаем **[Collection]** с тем же именем.

ФИКСТУРА И КОЛЛЕКЦИЯ

```
public class DatabaseFixture : IDisposable
{
    public DatabaseFixture()
    {
        Db = new SqlConnection("MyConnectionString");
    }
    public void Dispose()
    {
        // ... очистка тестовых данных в БД ...
    }
    public SqlConnection Db { get; private set; }
}

[CollectionDefinition("Database collection")]
public class DatabaseCollection
    : ICollectionFixture<DatabaseFixture> { }
```

ТЕСТОВЫЕ КЛАССЫ

```
[Collection("Database collection")]
public class DatabaseTestClass1
{
    DatabaseFixture fixture;
    public DatabaseTestClass1(DatabaseFixture fixture)
    {
        this.fixture = fixture;
    }
}

[Collection("Database collection")]
public class DatabaseTestClass2
{
    // ...
}
```

NUnit

Структура та же - меняются атрибуты и стиль Assert.

XUNIT

```
public class StringCalculatorTests
{
    private readonly StringCalculator _calculator;
    public StringCalculatorTests()
    {
        _calculator = new StringCalculator();
    }
    [Theory]
    [InlineData("0,0,0", 0)]
    [InlineData("1,2,3", 6)]
    public void Add_MultipleNumbers_ReturnsSum(
        string input, int expected)
    {
        var actual = _calculator.Add(input);
        Assert.Equal(expected, actual);
    }
}
```

NUNIT

```
public class StringCalculatorTests
{
    private StringCalculator _calculator;
    [SetUp]
    public void SetUp()
    {
        _calculator = new StringCalculator();
    }
    [TestCase("0,0,0", 0)]
    [TestCase("1,2,3", 6)]
    public void Add_MultipleNumbers_ReturnsSum(
        string input, int expected)
    {
        var actual = _calculator.Add(input);
        Assert.That(actual, Is.EqualTo(expected));
    }
}
```

xUnit ↔ NUnit

Что	xUnit	NUnit
Тестовый класс	обычный класс	обычный класс
Тест без параметров	[Fact]	[Test]
Тест с параметрами	[Theory] + [InlineData]	[TestCase(...)]
Данные из метода	[MemberData]	[TestCaseSource]
Перед каждым тестом	конструктор	[SetUp]
После каждого теста	Dispose()	[TearDown]
Один раз на класс	IClassFixture<T>	[OneTimeSetUp]
Проверка	Assert.Equal(exp, act)	Assert.That(act, Is.EqualTo(exp))

05

Assertions

Встроенный Assert и библиотека FluentAssertions

05 · ASSERTIONS

Assert в xUnit

Equal	Same
Contains	InRange
Empty	True / False
IsType<T>	Null
Throws<T>	Equivalent

- **Same** - по ссылке
- **Equivalent** - по значениям полей
- **IsAssignableFrom** - реализует ли тип/интерфейс

```
[Fact]
public void TestMethod_ShouldHandleException()
{
    // Arrange
    var exceptionType = typeof(InvalidOperationException);
    var expectedMessage = "The operation is invalid.";
    // Act
    var ex = Record.Exception(() => {
        throw new InvalidOperationException(expectedMessage);
    });
    // Assert
    Assert.NotNull(ex);
    Assert.IsType(exceptionType, ex);
    Assert.Equal(expectedMessage, ex.Message);
}
```

05 · ASSERTIONS

FluentAssertions

```
string actual = "ABCDEFGHI";
actual.Should().StartWith("AB").And.EndWith("HI").And.Contain("EF").And.HaveLength(9);

IEnumerable<int> numbers = new[] { 1, 2, 3 };
numbers.Should().OnlyContain(n ⟹ n > 0);
numbers.Should().HaveCount(4, "because we thought we put four items in the collection");

dictionary.Should().ContainValue(myClass).Which.SomeProperty.Should().BeGreaterThan(0);
someObject.Should().BeOfType<Exception>().Which.Message.Should().Be("Other Message");
xDocument.Should().HaveElement("child").Which.Should().BeOfType<XElement>().And.HaveAttribute("attr", "1");
```

Should()

начинает проверку

And / Which

цепочки проверок

because

пояснение в сообщении об ошибке

06

Моки

Moq и NSubstitute: настройка, возврат значений, проверка вызовов

06 · МОКИ

Mock-объекты и Moq

Mock-объект - фиктивная реализация интерфейса для тестирования взаимодействия; относительно него высказывается утверждение.

Ограничение

Moq мокает интерфейсы и виртуальные методы. Невиртуальные и статические - нельзя.

Loose (дефолт)

Вызов без Setup вернёт значение по умолчанию: 0, null...

Strict

Вызов без Setup - исключение. При небольших зависимостях берите Strict.

Moq: основной API

```
var mock = new Mock<ILoveThisLibrary>();

// Настраиваем поведение
mock.Setup(library ⇒ library.DownloadExists("2.0.0.0"))
    .Returns(true);

// .Object - реализация ILoveThisLibrary
ILoveThisLibrary lovable = mock.Object;
bool download = lovable.DownloadExists("2.0.0.0");

// Метод вызван не более одного раза
mock.Verify(library ⇒ library.DownloadExists("2.0.0.0"),
    Times.AtMostOnce());
```

`Setup`
какой метод и с какими аргументами

`Returns(Async)`
значение или лямбда от аргументов

`.Object`
сама реализация интерфейса

`Verify`
сколько раз и с какими аргументами

Тестовый класс с моками

1. Моки и общий Setup - в поля и конструктор
2. В конструкторе создаём тестируемый класс с mock.Object
3. В каждом тесте - свои Setup под сценарий
4. В конце - Verify вызовов зависимостей

ПРИМЕР

```
public class OrderServiceTests
{
    private readonly Mock<IOrderRepository> _repository =
        new(MockBehavior.Strict);
    private readonly OrderService _service;
    public OrderServiceTests()
    {
        _service = new OrderService(_repository.Object);
    }
    [Fact]
    public async Task Create_ValidOrder_SavesOrder()
    {
        _repository.Setup(r ⇒ r.SaveAsync(It.IsAny<Order>()))
            .ReturnsAsync(true);
        await _service.CreateAsync(new Order());
        _repository.Verify(r ⇒ r.SaveAsync(It.IsAny<Order>()),
            Times.Once);
    }
}
```

06 · МОКИ

NSubstitute

Альтернатива Moq, которая заметно набрала популярность в последние годы: многие проекты перешли на неё после истории с SponsorLink в Moq (2023).

Синтаксис короче: подмена - это **сам объект**, без лямбд в Setup и без .Object.

Режима Strict нет: все подмены «loose».

```
var repository = Substitute.For<IProductRepository>();

// настройка: вызываем метод и говорим, что вернуть
repository.Get(1).Returns(new Product { Id = 1 });
repository.Get(Arg.Any<long>()).Returns(new Product());

var service = new ProductService(repository);
var product = service.GetProduct(1);

product.Id.Should().Be(1);
// проверка вызовов
repository.Received(1).Get(1);
repository.DidNotReceive().Delete(Arg.Any<long>());
```

Moq vs NSubstitute

Действие	Moq	NSubstitute
Создать	new Mock<IRepo>()	Substitute.For<IRepo>()
Настроить	mock.Setup(r => r.Get(1)).Returns(p)	repo.Get(1).Returns(p)
Любой аргумент	It.IsAny<long>()	Arg.Any<long>()
Исключение	.Setup(...).Throws<Ex>()	repo.Get(1).Throws<Ex>()
Передать в код	mock.Object	repo
Был вызван	Verify(r => r.Get(1), Times.Once())	repo.Received(1).Get(1)
Не был вызван	Verify(..., Times.Never())	repo.DidNotReceive().Get(1)

07

Тестовые данные

Builder и фейкеры: Bogus, AutoBogus, AutoFixture

07 · ТЕСТОВЫЕ ДАННЫЕ

Откуда брать тестовые данные

Заполнять большие и сложные объекты руками в каждом тесте - долго и шумно. Есть два подхода:

Паттерн Builder

- Без сторонних зависимостей
- Для сложных доменных объектов
- Цепочка WithX(...), в конце Build()
- **Build() всегда возвращает валидный объект** - учитываем, например, взаимоисключающие флаги

Фейкеры

- Используются чаще
- Не нужно писать свои builder-ы
- **Bogus** - правила на каждое поле
- **AutoBogus** - автозаполнение + реалистичные данные
- **AutoFixture** - максимально простое автозаполнение

07 · ТЕСТОВЫЕ ДАННЫЕ

Bogus

Правило **RuleFor** на каждое поле - очень похоже на ручной builder, но писать его не нужно. Данные реалистичные: имена, email, адреса.

ПРАВИЛА ДЛЯ ПОЛЕЙ

```
var expectedProduct = new Faker<Product>()
    .RuleFor(p ⇒ p.Id, f ⇒ f.IndexFaker)
    .RuleFor(p ⇒ p.WarehouseId, f ⇒ f.IndexFaker)
    .RuleFor(p ⇒ p.Name, f ⇒ f.Name.FirstName())
    .Generate();                          // или .Generate(10) - список
```

CUSTOMINSTANTIATOR

```
// нет конструктора без параметров - создаём объект сами
var product = new Faker<ProductDto>()
    .CustomInstantiator(f ⇒ new ProductDto(f.IndexFaker, f.Name.FullName()))
    .Generate();
```

07 · ТЕСТОВЫЕ ДАННЫЕ

AutoBogus

Расширение Bogus (подтягивает его сам): заполняет **все поля автоматически**, а RuleFor по-прежнему доступен для нужных полей.

```
// пакет AutoBogus.Conventions
AutoFaker.Configure(f ⇒ f.WithConventions());

var product = new AutoFaker<Product>().Generate();

// точечно переопределяем поле
var withId = new AutoFaker<Product>()
    .RuleFor(p ⇒ p.Id, f ⇒ 1)
    .Generate();
```

ЧТО СГЕНЕРИРОВАЛОСЬ

Email
`"Ivy_Hauck81@hotmail.com"`

CompanyName
`"Padberg, Wuckert and Pouros"`

Name
`"implementation"`

Id
`-81284642927367318 95`

Конвенции: поле Email получает похожий на настоящий адрес.

07 · ТЕСТОВЫЕ ДАННЫЕ

AutoFixture

Заполняет объект **максимально простыми**, нереалистичными значениями. Цель - быстро получить любые валидные данные.

FIXTURE

```
var fixture = new Fixture();
var product = fixture.Create<Product>();
// Id = 121, WarehouseId = 224
// Name = "Nameda99cc86-d474-494f-..."
```

[AUTODATA]

```
// пакет AutoFixture.Xunit2
[Theory, AutoData]
public void Get_ShouldReturnProduct(
    Product product)
{
    // product уже заполнен
}
```

[AutoData]

объекты-параметры теста создаются автоматически

AutoMoq / AutoNSubstitute

кастомизации, которые подставляют моки вместо интерфейсов

Что выбрать

Подход	Как задаём данные	Данные	Когда брать
Builder	Свой класс, Build()	Любые, всегда валидные	Сложные доменные объекты, без зависимостей
Bogus	RuleFor на каждое поле	Реалистичные	Нужен контроль над значениями
AutoBogus	Автоматически + RuleFor	Реалистичные по конвенциям	Много полей и нужен реализм
AutoFixture	Автоматически	Простые, нереалистичные	Быстро получить любые значения, [AutoData]

Заключение

- ✓ Основа пирамиды - быстрые и независимые unit-тесты на бизнес-логику
- ✓ Имя: метод_сценарий_результат; структура AAA (Given / When / Then)
- ✓ Один тест - один сценарий; никакой логики и «магии» в тестах
- ✓ Время, БД и сервисы - через абстракции и моки (Moq, NSubstitute)
- ✓ Тестовые данные - Builder или фейкеры: Bogus, AutoBogus, AutoFixture